Sofia University
Department of Mathematics and Informatics

<u>Course</u> : OO Programming C#.NET
<u>Date</u>: March 4, 2026
<u>Student</u> Name:

Домашно 2

Submit the all C# .NET files developed to solve the problems listed below. Use comments and Modified-Hungarian notation.

<u>Problem No. 1</u>

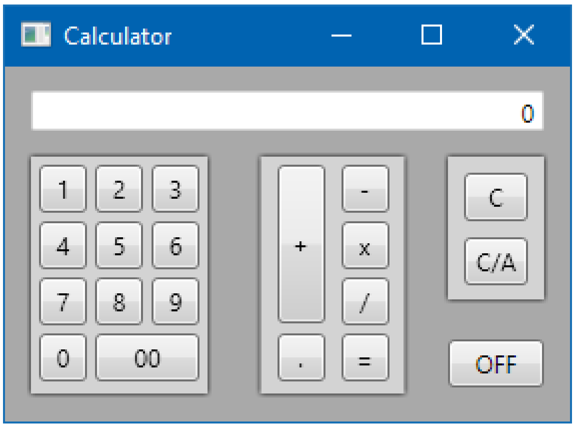
Write a **WPF application** to implement the **functions** and **user interface** of a **calculator** i.e. there should be:

a) all the arithmetic operations

b) memory store, clear, add, subtract

c) mathematical functions for `EXP()`, `SIN()`, `COS()`, `SQRT()`, `LOG()` and `1/x`

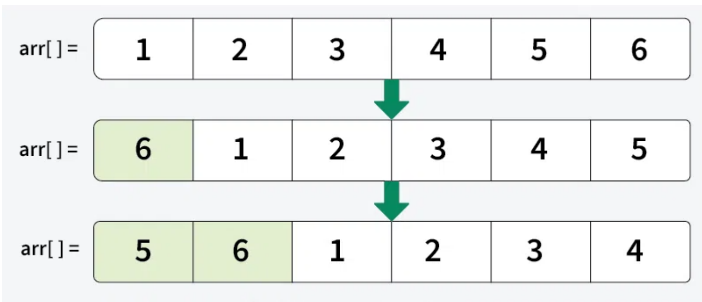
Make use of `enum` types to denote arithmetic and memory operations. Accordingly, use these `enum` constants in a switch command to execute the calculator operations.



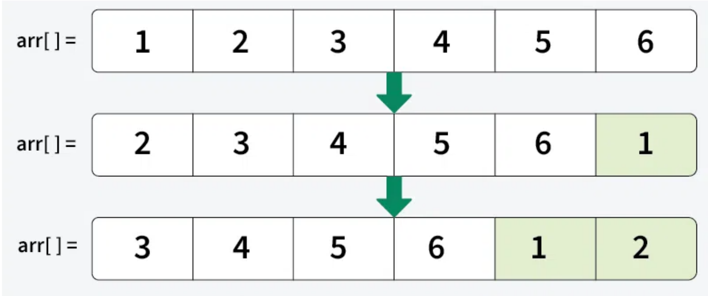
<u>Problem No. 2</u>

Array rotation involves rearranging the elements of an array by shifting them either to the left (counterclockwise) or to the right (clockwise). The number of positions to rotate is specified, and the elements wrap around to the other end of the array. One approach to rotate is to **rotate one by one**. This approach shifts elements one position at a time in a circular manner. For a right rotation by **d** positions, this process is repeated **d** times.

Example:
Input arr = [**1, 2, 3, 4, 5, 6**], d = **2** Output: [**5, 6, 1, 2, 3, 4**]
Input arr = [**1, 2, 3, 4, 5, 6, 7]** , d = **3**, Output: [**5, 6, 7, 1, 2, 3, 4**]
Rotate right



Rotate left



The **reversal algorithm** involves reversing parts of the array to achieve the rotation. First, reverse the entire array. Then reverse the first d elements and the remaining n-d elements separately.
Implement to **Reversal algorithm** to **write** a **Console application** to **rotate right** and **left** by k positions of the elements of a single dimensional array **without using additional memory** for the rotation commands. Declare the rotation orientation as **enum** type.

Hint: Use the **Span<>** data type